Senior to Staff Data Engineer: My 15-Week Plan

YouTube script, Notion screen-share version. Length: about 16.5 minutes at 150 words a minute.

Opening

[On camera]

I'm a senior data engineer at Meta, where the average base salary at my level is about two hundred twenty thousand dollars a year. The staff-level data jobs I'm going after at AI companies pay three hundred fifty to four hundred thousand in base salary, and some pay more. So I built a fifteen-week plan to make that jump, and I'm in week four right now. In this video, I'm going to open the exact Notion page I'm using and walk you through it week by week. You'll see what I study, the problems I solve, the system designs I practise, the project I'm building, and the three real jobs I'm preparing for. You can copy all of it.

[Card beside you: c1-opening-pay.png]

Intro

[Pause] [Intro motion graphic: intro-blue.mp4]

If you are new here, I am Chris. I have worked in data engineering for more than ten years. I have interviewed more than two hundred engineers for companies in healthcare, banking, automotive, and financial exchanges. Today I run Stackle, where I teach data and AI engineering, and more than two hundred thousand people follow me on LinkedIn.

[Pause]

Part 1: The one idea behind the plan

[On camera]

So before I open the page, here's the one idea the whole plan is built on. After years on the hiring side, the simplest way I can put it is this. Senior gets you hired for what you can build. Staff gets you hired for what you can design.

[Pause] [Graphic 1]

A senior data engineer takes a hard problem and builds it well, like a pipeline that moves a billion rows a day without breaking. A staff data engineer decides how all the pipelines should fit together, across many teams, before anyone writes the code. Dropbox even writes this into its published career levels. Its staff data engineer level says, "I set the multi-year, multi-team technical and data strategy."

[Pause] [Graphic 3]

And so my plan has a clear order. Number one is system design, because that's the skill that changes the most between senior and staff. Number two is coding, because you still have to pass the coding rounds. And the way I plan to stand out is data and AI infrastructure, because the AI companies are hiring data engineers to build the pipelines that feed their models.

[Pause]

Part 2: How the page works

[Screen share: the top of the Notion page, "The targets by the end of week 15"]

So here's the page. At the top are my targets for the end of week fifteen. That's a hundred fifty DSA problems, forty-seven advanced SQL problems, thirty Python questions, sixteen system designs, and twenty-five debugging scenarios. It's also ten stories from my own work, and one real project, which is a distributed AI platform.

[Screen share: "How I'm going to do it"]

Right under that is how I'm actually going to do it, because that's the part most plans leave out. It's about twenty hours a week. Every weekday morning, I do two DSA problems. I give each one twenty-five minutes, then I watch the solution, then I write a two-line note on the pattern. Weekday evenings are for the week's topic. Saturday is the system design of the week, forty-five minutes, out loud, with a timer, and then project work. And Sunday is debugging write-ups, two behavioral answers, and pushing everything to GitHub.

[On camera]

Now let's say a DSA problem takes you forty-five minutes. Two a day is an hour and a half, which is seven and a half hours a week on DSA alone. For comparison, a 2025 study of one hundred thirty-one people preparing for tech interviews found the average was

about six and a half hours a week, in total. So yes, this plan is more than most people do. That's the point.

Part 3: The fifteen weeks

[Pause] [Graphic 4]

So let's go week by week. I've split the fifteen weeks into four blocks: the coding base, data systems, AI, and getting ready to perform. And every week on the page has the same parts. There's what I learn and who I learn it from, the DSA and SQL problems, the Python questions, the system design, the debugging scenarios, the behavioral questions, and the project step.

[Pause]

Block one: The coding base

[Screen share: Week 1 open]

Week one is Python, SQL, and DSA. DSA just means data structures and algorithms, which are the coding puzzles big companies still ask. This week is arrays, hashing, and two pointers, with fifteen problems like Two Sum and Top K Frequent Elements, plus eight SQL problems on ranking. The Python questions are the data engineer kind, like reading a ten-gigabyte log file with a generator so it never has to fit in memory. And I start the dsa-practice repo on GitHub, where every problem gets a short note.

[Screen share: Week 2 open]

Week two is sliding window, stacks, and binary search, plus pandas, plus SQL running totals and streaks. One Python question I like here is sessionizing clicks. Sessionizing just means starting a new session after thirty minutes with no activity, and that shows up in real data jobs all the time. I also pick the ten moments from my own work that will become my interview stories.

Block two: Data systems

[Screen share: Week 3 open]

Week three is distributed systems. A distributed system just means many machines working as one, so if one machine dies, the work keeps going. I'm learning replication, partitioning, and consistency from Martin Kleppmann's lectures. He wrote Designing Data-Intensive Applications. My first system design is a real-time event pipeline. And the project starts for real, with Kafka running on my laptop and a producer that reads every edit made on Wikipedia, live.

[Screen share: Week 4 open]

Week four is Kafka, and that's where I am right now. Kafka is a tool that moves streams of events between systems as they happen. The design this week is clickstream ingestion at millions of events per second. The debugging scenarios are the classics: consumer lag that keeps growing, and duplicate events showing up downstream. And so my Kafka project has to handle the same event arriving twice, and keep running when I kill it halfway through a batch.

[Screen share: Week 5 open]

Week five is Spark, which is a tool that processes huge amounts of data by splitting the work across many machines. This week is how Spark actually runs a job, through stages, tasks, and shuffles. The design is a batch pipeline over billions of rows. And the debugging scenario is one every data engineer has seen, where one task runs for forty minutes while the rest finish in one. That's data skew, which just means one key has way more data than the others.

[Screen share: Week 6 open]

Week six is Spark performance. That means reading query plans, fixing skew, and using broadcast joins. The design is a streaming job with exactly-once output, which means every event counts once, even after a crash. And the project gets a streaming job that takes events from Kafka into the first layer of my tables.

[Screen share: Week 7 open]

Week seven is data modeling. That's star schemas, facts and dimensions, and slowly changing dimensions, which is how you keep the history when, let's say, a customer moves to another city. The designs are an Airbnb bookings warehouse and a Netflix playback pipeline.

[Screen share: Week 8 open]

Week eight is the lakehouse. A lakehouse just means warehouse-style tables sitting on cheap cloud storage, using table formats like Iceberg and Delta. The designs are Uber's pricing data and a metrics layer for a whole company. And the debugging this week is the scary kind, where nothing fails. Row counts drop thirty percent overnight, or the dashboard numbers double after a rerun, and no alarm goes off.

[Screen share: Week 9 open]

Week nine is AWS and Kubernetes. AWS is Amazon's cloud. Kubernetes runs your programs across many machines and restarts them when they crash. This is the week

the whole project moves onto Kubernetes, and I write up how it would run on AWS and on Snowflake. I'll show you those diagrams in a minute.

Block three: AI

[Screen share: Week 10 open]

Week ten is machine learning basics, which is how models learn from data, like an inbox that learns which emails are spam. For a data engineer, the part that matters most is features. Features just means the inputs a model learns from, and the hard part is making sure training and production use the same ones. The designs are a feature pipeline for recommendations and a real-time fraud score.

[Screen share: Week 11 open]

Week eleven is AI engineering, which means building products on top of large language models, the models behind ChatGPT, Claude, and Gemini. I'm learning embeddings, vector databases, and RAG. RAG just means the model looks things up in your documents before it answers. The designs are a RAG system over company documents and an AI agent with tools. And my project gets its AI layer, so you can ask what changed on Wikipedia today and get an answer that cites the articles.

[Screen share: Week 12 open]

Week twelve is AI infrastructure, and this is where a data engineer really fits in. AI infrastructure just means the data systems around the model: the pipelines that feed it, the systems that serve it, and the checks that stop bad data from reaching it. The designs are an LLM inference platform and a training data pipeline. And by the end of this week, the project is finished, with a thirty-question evaluation set, a monitoring dashboard, and a one-page design doc.

Block four: Getting ready to perform

[Screen share: Week 13 open]

Week thirteen is debugging. I write up all twenty-five scenarios the same way: the symptom, the cause, how to confirm it, the fix, and how to stop it from happening again. And I practise the question senior interviewers love most: a pipeline that took one hour now takes three, so walk me through it. I'll also break my own project on purpose, five times, and debug each one.

[Screen share: Week 14 open]

Week fourteen is mock interviews. I redo every design out loud, forty-five minutes each, plus timed coding rounds. And I'm adding one round where I'm allowed to use AI,

because some big companies, Meta included, have started testing coding interviews where you can use an AI assistant.

[Screen share: Week 15 open]

And week fifteen is behavioral and applying. I finish my ten stories, I finish my resume, and I apply.

[On camera]

Now, fifteen weeks is tight. If something slips, I'm fine stretching the plan to twenty. The order matters more than the dates.

Part 4: The architecture

[Screen share: "Architecture", the core design diagram]

Now let me show you the project, because this is the part that turns a study plan into proof. The data comes from Wikipedia's live stream of edits, which is free and never stops. Kafka brings it in as it happens, and Spark cleans it and removes duplicates. It lands in Iceberg tables in three layers: raw, cleaned, and ready to use. Then an embedding job turns the articles into something a model can search, and an API answers your questions with sources. And all of it runs on Kubernetes.

[Screen share: "2. The AWS version"]

Then I show the same design two more ways. On AWS, Kafka becomes Amazon MSK, Spark runs on EMR, the tables sit in S3, and the model comes from Amazon Bedrock. One warning, though. Some of those services charge while they're running, even when nobody's using them, so I build it, record a demo, and tear it down the same day.

[Screen share: "3. The Snowflake version"]

On Snowflake, the data streams straight in with Snowpipe Streaming, which is Snowflake's tool for loading data as it arrives. Then dbt builds the layers, and Snowflake's Cortex handles the search and the model.

[Screen share: "Which version to talk about in an interview"]

And so in an interview, I can talk about both. The AWS version shows platform depth. The Snowflake version shows modeling and speed. Being able to compare them, and say why I'd pick one, is exactly the kind of decision a staff engineer gets asked about.

Part 5: The three jobs, and my resume

[Screen share: "The three jobs I'm preparing for"]

Next are the three jobs I'm preparing for. They're linked right here, and the pay ranges are copied straight from the postings. They're base salary, which just means the pay before stock and bonus.

[Pause] [Graphic 11]

The first job is at Anthropic. It's called Staff Plus Software Engineer, Data Infrastructure, and the base salary is four hundred five to four hundred eighty-five thousand dollars a year. It says you should be able to "set technical direction for a team, not just execute within it." That one line is the difference between senior and staff, and the company wrote it down for you.

[Pause] [Graphic 12]

The second job is at OpenAI, and it's called Software Engineer, Data Infrastructure. The range is two hundred sixty-six to four hundred forty-five thousand, with equity on top. It talks about huge Spark systems, streaming on Kafka and Flink, and debugging large distributed systems. So that's weeks three to six and week thirteen of my plan.

[Pause] [Graphic 13]

The third job is at Google. It's called Staff Software Engineer, AI Foundry Data Quality and Training Data. The base is two hundred seven to three hundred thousand, plus a twenty percent bonus target and equity. The job is cleaning training data for large language models, which is week twelve. It also asks for two years of leading people, so read every line of a posting before you apply.

[Pause]

[On camera]

And notice that two of those three jobs aren't called "data engineer" at all. At the staff level, a lot of data jobs are called "software engineer, data infrastructure." If you only search for "data engineer," you'll miss them.

[Screen share: "Resume", the requirement-to-proof table]

So this is how I'm rewriting my resume. For every requirement in those postings, I need one line of proof. If there's no proof, that's a gap, and the plan has a week for it. And every line has to show a decision, not just a build.

[Pause] [Graphic 15]

So let's say your resume says, "Built a Spark pipeline that processes two terabytes a day." That's a good senior line. A staff line sounds more like, "Designed a shared data model used by four teams, which removed three duplicate pipelines." It's the same engineer with the same skills. But the second line shows the decision and the reach.

[Pause]

Part 6: Getting your stories right

[Screen share: "Behavioral questions and the story bank"]

The last part is the one I think most people miss, which is getting your stories right. A behavioral round just means the interview where they ask about things you've actually done, like "Tell me about a time you disagreed with another team." Amazon's own interview guide says each interviewer will typically ask two or three of these questions. So the page has twenty-four questions, two a week, and ten story slots.

[Pause] [Graphic 16]

Every story gets the same five parts: the situation, the decision I made, the trade-off, the result, and what I'd do differently. A weak story is about what happened. A strong story is about what you decided, and why. And each story should answer two or three different questions, so ten deep stories can cover almost anything they ask.

[Pause]

Your turn

[On camera]

So here's your one task for this week. Write down one story from your own work, using those five parts. It'll take about thirty minutes. Most people won't do it. They think they'll remember the story in the interview, and then they ramble. If you write it down now, you're already ahead of them.

[OPTIONAL, only if you publish a copy of the page]

And if you want this exact page as a template, there's a copy linked in the description.

Call to action

[Pause] [CTA motion graphic: cta-blue.mp4]

If you want help building your own plan, book a free fifteen-minute career call with me. The link is in the description. And if you want a structured path with real projects and feedback, the Stackle Data Engineering Masterclass is linked there too.

[Pause]

[On camera]

I'm in week four right now. Tell me in the comments: which week of this plan would be the hardest for you?